Class: SE **Subject: COA LAB** **Sem-III**

Experiment No 5

Course Outcome: CO3 **Blooms Level: L3**

Aim: Implementation of Block Transfer and Block Exchange using Index Registers.

Abstract:

1)Develop an 8086 assembly language program to copy a block of five bytes from a source memory location to a destination memory location. The source data is defined within a data segment, and the destination starts a few bytes after the end of the source block. The program should transfer all five bytes correctly using appropriate register-based addressing and looping techniques. The source and destination addresses must be handled using pointers, and the number of bytes to be transferred should be controlled by a counter register.

2)Design an 8086 assembly language program to exchange the contents of two memory blocks of equal size. Each block contains five bytes of predefined data stored in the data segment. The program must swap the corresponding bytes of the two blocks such that the contents of the first block are moved to the second block and vice versa.

Sample Input and Output:

Case 1:Block Transfer
Overlapping block transfer
Input: 01 02 03 04 05
Output: 01 02 03 01 02 03 04 05
Non-overlapping block transfer
Input: 01 02 03 04 05
Output: 01 02 03 04 05 01 02 03 04 05

Case 2:Block Exchange
Input: 01 02 03 04 05 11 12 13 14 15
Output: 11 12 13 14 15 01 02 03 04 05

Theory:

Block operations in 8086 assembly language are essential for efficiently handling sequences of data in memory. Two common types of block operations are block transfer and block exchange.

- Block Transfer:

Block transfer refers to copying a sequence of bytes from one memory location (source) to another (destination). In this operation, the original data remains unchanged at the source, and a duplicate is created at the target location. This is typically achieved using instructions such as MOV in a loop,

with registers like SI (source index), DI (destination index), and CX (counter) controlling the process. The addresses are updated using INC or DEC, depending on whether the transfer is forward or backward. Block transfer is useful in applications like data backup, memory initialization, and buffer duplication.

- Block Exchange:

Block exchange involves swapping the contents of two memory blocks of equal size. Unlike transfer, in exchange operations, both memory areas are modified — each byte from the first block is swapped with its corresponding byte in the second block. This process typically uses temporary registers like AL and BL for holding data during the exchange, along with SI and DI to point to the respective blocks. Block exchange is commonly used in sorting algorithms, memory shuffling, and data manipulation routines that require in-place updates without additional memory.

Program:

Block Transfer:

```
.model small
.data
x db 01h,02h,03h,04h,05h
.code
start:
mov ax,@data
mov ds,ax
mov cx,05h
lea si,x+04
lea di,x+04+03
up: mov bl,[si]
mov [di],bl
dec si
dec di
dec cx
jnz up
mov ah,4ch
int 21h
end start
```

Class: SE **Subject: COA LAB** **Sem-III**

Block Exchange:

```
.MODEL SMALL
.DATA
block1 DB 01h, 02h, 03h, 04h, 05h
block2 DB 11h, 12h, 13h, 14h, 15h

.CODE
START:
    MOV AX, @DATA
    MOV DS, AX

    MOV CX, 05h          ; Number of bytes to exchange
    LEA SI, block1       ; Source block 1
    LEA DI, block2       ; Source block 2

EXCHANGE_LOOP:
    MOV AL, [SI]         ; Load byte from block1
    MOV BL, [DI]         ; Load byte from block2

    MOV [SI], BL         ; Store block2's byte into block1
    MOV [DI], AL         ; Store block1's byte into block2

    INC SI
    INC DI
    DEC CX
    JNZ EXCHANGE_LOOP

    MOV AH, 4Ch
    INT 21h
END START
```

Output:

Before Block Transfer:

```
C:\>DEBUG EXP5.EXE
-T

AX=076B  BX=0000  CX=0023  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=0769  CS=076A  IP=0003   NV UP EI PL NZ NA PO NC
076A:0003 8ED8          MOV     DS,AX
-T

AX=076B  BX=0000  CX=0023  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=076B  ES=075A  SS=0769  CS=076A  IP=0005   NV UP EI PL NZ NA PO NC
076A:0005 B90500        MOV     CX,0005
-D DS:0
076B:0000  8A 1C 88 1D 4E 4F 49 75-F7 B4 4C CD 21 00 01 02   ....NOIu..L.!...
076B:0010  03 04 05 C4 04 50 E8 9F-0E 83 C4 04 3D FF FF 74   .....P......=..t
076B:0020  03 E9 11 01 B8 2F 00 50-8B 46 FC 8B 56 FE 05 0C   ...../.P.F..V...
076B:0030  00 52 50 E8 EA 48 83 C4-04 50 E8 7B 0E 83 C4 04   .RP..H...P.{....
076B:0040  3D FF FF 74 03 E9 ED 00-C4 5E FC 26 8A 47 0C 2A   =..t.....^.&.G.*
076B:0050  E4 40 50 8B C3 8C C2 05-0C 00 52 50 E8 C1 48 83   .@P.......RP..H.
076B:0060  C4 04 50 8D 86 FA FE 50-E8 17 73 83 C4 06 8B B6   ..P....P..s.....
076B:0070  FA FE 81 E6 FF 00 C6 82-FB FE 00 2B C0 50 8D 86   ...........+.P..
-
```

After Block Transfer:

```
AX=076B  BX=0001  CX=0000  DX=0000  SP=0000  BP=0000  SI=000D  DI=0010
DS=076B  ES=075A  SS=0769  CS=076A  IP=0019   NV UP EI PL ZR NA PE NC
076A:0019 B44C          MOV     AH,4C
-D DS:0
076B:0000  8A 1C 88 1D 4E 4F 49 75-F7 B4 4C CD 21 00 01 02   ....NOIu..L.!...
076B:0010  03 01 02 03 04 05 E8 9F-0E 83 C4 04 3D FF FF 74   ............=..t
076B:0020  03 E9 11 01 B8 2F 00 50-8B 46 FC 8B 56 FE 05 0C   ...../.P.F..V...
076B:0030  00 52 50 E8 EA 48 83 C4-04 50 E8 7B 0E 83 C4 04   .RP..H...P.{....
076B:0040  3D FF FF 74 03 E9 ED 00-C4 5E FC 26 8A 47 0C 2A   =..t.....^.&.G.*
076B:0050  E4 40 50 8B C3 8C C2 05-0C 00 52 50 E8 C1 48 83   .@P.......RP..H.
076B:0060  C4 04 50 8D 86 FA FE 50-E8 17 73 83 C4 06 8B B6   ..P....P..s.....
076B:0070  FA FE 81 E6 FF 00 C6 82-FB FE 00 2B C0 50 8D 86   ...........+.P..
```

Before Block Exchange:

```
C:\>debug exp52.exe
-t

AX=076C  BX=0000  CX=002C  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=075A  ES=075A  SS=0769  CS=076A  IP=0003   NV UP EI PL NZ NA PO NC
076A:0003 8ED8          MOV     DS,AX
-t

AX=076C  BX=0000  CX=002C  DX=0000  SP=0000  BP=0000  SI=0000  DI=0000
DS=076C  ES=075A  SS=0769  CS=076A  IP=0005   NV UP EI PL NZ NA PO NC
076A:0005 B90500        MOV     CX,0005
-d ds:0
076C:0000  21 00 01 02 03 04 05 11-12 13 14 15 3D FF FF 74   !...........=..t
076C:0010  03 E9 11 01 B8 2F 00 50-8B 46 FC 8B 56 FE 05 0C   ...../.P.F..V...
076C:0020  00 52 50 E8 EA 48 83 C4-04 50 E8 7B 0E 83 C4 04   .RP..H...P.{....
076C:0030  3D FF FF 74 03 E9 ED 00-C4 5E FC 26 8A 47 0C 2A   =..t.....^.&.G.*
076C:0040  E4 40 50 8B C3 8C C2 05-0C 00 52 50 E8 C1 48 83   .@P.......RP..H.
076C:0050  C4 04 50 8D 86 FA FE 50-E8 17 73 83 C4 06 8B B6   ..P....P..s.....
076C:0060  FA FE 81 E6 FF 00 C6 82-FB FE 00 2B C0 50 8D 86   ...........+.P..
076C:0070  FB FE 50 E8 08 6A 83 C4-04 0B C0 75 03 E9 A5 00   ..P..j.....u....
-_
```

After Block Exchange:

```
AX=0705  BX=0015  CX=0000  DX=0000  SP=0000  BP=0000  SI=0007  DI=000C
DS=076C  ES=075A  SS=0769  CS=076A  IP=001D   NV UP EI PL ZR NA PE NC
076A:001D B44C          MOV     AH,4C
-d ds:0
076C:0000  21 00 11 12 13 14 15 01-02 03 04 05 3D FF FF 74   !...........=..t
076C:0010  03 E9 11 01 B8 2F 00 50-8B 46 FC 8B 56 FE 05 0C   ...../.P.F..V...
076C:0020  00 52 50 E8 EA 48 83 C4-04 50 E8 7B 0E 83 C4 04   .RP..H...P.{....
076C:0030  3D FF FF 74 03 E9 ED 00-C4 5E FC 26 8A 47 0C 2A   =..t.....^.&.G.*
076C:0040  E4 40 50 8B C3 8C C2 05-0C 00 52 50 E8 C1 48 83   .@P.......RP..H.
076C:0050  C4 04 50 8D 86 FA FE 50-E8 17 73 83 C4 06 8B B6   ..P....P..s.....
076C:0060  FA FE 81 E6 FF 00 C6 82-FB FE 00 2B C0 50 8D 86   ...........+.P..
076C:0070  FB FE 50 E8 08 6A 83 C4-04 0B C0 75 03 E9 A5 00   ..P..j.....u....
-_
```

Class: SE **Subject: COA LAB** **Sem-III**

Conclusion:
The experiment was successfully performed using 8086 assembly language. It demonstrated how to efficiently move and exchange blocks of data in memory using index registers—SI (Source Index) and DI (Destination Index)—along with loop counters and string manipulation instructions.

In the block transfer operation, data was copied from one memory block to another using instructions like MOVSB, with the CX register controlling the number of bytes transferred. In the block exchange, corresponding elements from two memory blocks were swapped using a temporary register and iterative logic.

Exercise 1:
Aim: To write an assembly language program to sort an array of numbers.

Abstract: Develop an 8086 assembly language program to sort an array of five ASCII digits stored in memory. The program must take the unsorted data, arrange it in ascending order, and display the sorted result on the screen.